HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA KỸ THUẬT ĐIỆN TỬ 1

=====o0o=====

BÁO CÁO BÀI TẬP LỚN

Kỹ thuật Vi xử lý

Đề bài: "Thiết kế mô hình Automotive Gateway cho xe điều khiển phân tán: Tích hợp kiến trúc cập nhật phần mềm không dây tự phục hồi (OTA Dual-Bank Rollback) qua mạng CAN Bus "

Giảng viên hướng dẫn : TS. Nguyễn Trung Hiếu

Sinh viên thực hiện : Phạm Quốc Trung - B24DCVT361

: Dương Bảo Long - B24DCVT231

Hệ đào tạo : Đại học chính quy

Hà Nội - 2026

LỜI CẢM ƠN

Để hoàn thành bài tập lớn môn Kỹ thuật Vi xử lý với đề tài với đề tài tích hợp hệ thống điều khiển và truyền thông, nhóm chúng em đã nhận được sự hướng dẫn và hỗ trợ từ quý Thầy/Cô trong Học viện Công nghệ Bưu chính Viễn thông.

Trước tiên, nhóm nghiên cứu xin gửi lời cảm ơn trân trọng tới các Thầy/Cô giảng dạy học phần Kỹ thuật Vi xử lý thuộc Khoa Viễn thông 1. Các kiến thức nền tảng về tổ chức vi điều khiển, quản lý thanh ghi trực tiếp, xử lý ngắt thời gian thực và ghép nối bus truyền thông ngoại vi là cơ sở lý thuyết và kỹ thuật quan trọng giúp nhóm hoàn thành đề tài.

Đặc biệt, nhóm xin bày tỏ lòng cảm ơn sâu sắc tới Thầy/Cô phụ trách học phần đã định hướng đề tài sát với thực tiễn kỹ thuật công nghiệp ô tô (*Software-Defined Vehicles*), giải đáp các vấn đề chuyên môn và hướng dẫn phương pháp lập trình nhúng trực tiếp cấp thanh ghi cũng như quy chuẩn hóa giải thuật tự phục hồi firmware Dual-Bank.

Mặc dù đã cố gắng hoàn thiện các nội dung theo yêu cầu kỹ thuật, báo cáo khó tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những ý kiến đóng góp từ Thầy/Cô để đề tài được hoàn thiện hơn.

Nhóm sinh viên thực hiện xin chân thành cảm ơn!

Mục lục

LỜI CẢM ƠN....1
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT....6
DANH MỤC HÌNH VẼ....8
LỜI MỞ ĐẦU....1
1. Bối cảnh và tính cấp thiết của đề tài....1
2. Mục tiêu kỹ thuật của đề tài....1
3. Đối tượng và phạm vi nghiên cứu....2
4. Bố cục của báo cáo....3
CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG MẠNG TRUYỀN THÔNG Ô TÔ VÀ CÔNG NGHỆ FOTA....4
1.1. Kiến trúc điều khiển phân tán trên ô tô hiện đại....4
1.1.1. Sự phát triển của các khối điều khiển điện tử (ECU) trong xe hơi....4
1.1.2. Vai trò trung tâm của Automotive Gateway....4
1.2. Công nghệ cập nhật phần mềm không dây (FOTA - Firmware Over-The-Air)....5
1.2.1. Khái niệm và nguyên lý hoạt động tổng quát của FOTA....5
1.2.2. Các thách thức kỹ thuật cốt lõi trong môi trường nhúng ô tô....6
1.3. Giao thức mạng truyền thông Controller Area Network (CAN Bus)....6
1.3.1. Đặc tính vật lý và cấu trúc khung truyền CAN 2.0A....6
1.3.2. Cơ chế phân xử phi phá hủy (CSMA/CD + NBA)....7
1.4. Ý nghĩa ứng dụng của kiến thức Vi xử lý và Viễn thông trong đề tài....8
1.5. Kết luận chương 1....9
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH LỰA CHỌN GIẢI PHÁP THIẾT KẾ....10
2.1. Phân tích bài toán kỹ thuật và yêu cầu an toàn hệ thống....10
2.1.1. Mô tả bài toán xe điều khiển phân tán 2 cấp....10
2.1.2. Yêu cầu an toàn tiền điều kiện (Pre-condition Safety Check)....10
2.2. Phân tích và lựa chọn nền tảng phần cứng....11

2.2.1. So sánh và lựa chọn vi điều khiển 11
2.2.2. Lựa chọn các mô-đun ngoại vi và cơ cấu chấp hành 11
2.3. Cơ sở lý thuyết về quản lý bộ nhớ Flash và kiến trúc Dual-Bank tự phục hồi ..12
2.3.1. So sánh kiến trúc Single-Bank và Dual-Bank Rollback 12
2.3.2. Bản đồ phân vùng bộ nhớ Flash STM32F103 (128KB Flash)..... 13
2.3.3. Cơ chế hoán đổi nguyên tử (Atomic Swap) qua Option Bytes Data0 14
2.4. Thiết kế giao thức truyền thông phân luồng qua mạng CAN Bus 15
2.4.1. Kiến trúc phân bổ trường bit trong định danh CAN ID (11-bit Standard ID) 15
2.4.2. Bảng định danh thông điệp điều khiển thời gian thực (Control Stream - FIFO 0) 17
2.4.3. Bảng quy hoạch và đặc tả khung truyền nạp Firmware (OTA Stream - FIFO 1) 20
2.4.4. Giải thuật truyền khối trượt (Burst Chunk Transmission) và phát hiện sai lỗi 23
2.5. Công cụ phát triển, biên dịch trần Makefile và gỡ lỗi phần cứng 24
2.6. Kết luận chương 2 25
CHƯƠNG 3: THIẾT KẾ CHI TIẾT, HIỆN THỰC HÓA VÀ THỰC NGHIỆM ĐÁNH GIÁ HỆ THỐNG 26
3.1. Thiết kế kiến trúc tổng thể của hệ thống 26
3.1.1. Sơ đồ khối chi tiết các mô-đun phần cứng 26
3.1.2. Sơ đồ nguyên lý kết nối chi tiết (Wiring Diagram)..... 26
3.2. Thiết kế và hiện thực hóa phần mềm trên ESP32 Gateway 28
3.3. Thiết kế và hiện thực hóa phần mềm trên STM32 ECU (Không dùng HAL) ...29
3.4. Toàn trình quy trình cập nhật OTA từng bước (Step-by-Step Sequence)..... 31
3.5. Triển khai kịch bản kiểm thử và kết quả thực nghiệm 32
3.6. Đánh giá hiệu năng và thảo luận kết quả..... 34
3.6.1. Đánh giá thời gian nạp và thông lượng truyền thông..... 34
3.6.2. Đối chiếu chỉ tiêu kỹ thuật với yêu cầu đề bài 35
3.7. Kết luận chương 3 35

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 37
1. Các kết quả chính đã hoàn thành của đề tài 37
2. Các mặt còn hạn chế của mô hình 37
3. Hướng mở rộng và phát triển tiếp theo 38
TÀI LIỆU THAM KHẢO 39

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Ký hiệu / Chữ viết tắt	Từ ngữ Tiếng Anh đầy đủ	Diễn giải nghĩa Tiếng Việt
ACK	Acknowledgment	Tín hiệu xác nhận đã nhận dữ liệu thành công
BKP	Backup Registers	Vùng thanh ghi lưu trữ dữ liệu dự phòng trên STM32
BTL	Bài tập lớn	Bài tập lớn học phần đại học
bxCAN	Basic Extended Controller Area Network	Bộ điều khiển mạng CAN phần cứng tích hợp trên STM32
CAN	Controller Area Network	Mạng khu vực điều khiển (truyền thông vi sai 2 dây)
CRC	Cyclic Redundancy Check	Mã kiểm tra phần dư tuần hoàn (phát hiện lỗi bit)
DOF	Degree of Freedom	Bậc tự do (chuyển động cơ học)
ECU	Electronic Control Unit	Khối điều khiển điện tử trên phương tiện
FOTA / OTA	Firmware Over-The-Air	Cập nhật phần mềm nhúng từ xa qua mạng không dây
HAL	Hardware Abstraction Layer	Lớp trừu tượng hóa phần cứng (thư viện cấp cao)
HUD	Heads-Up Display	Màn hình hiển thị thông tin trực quan
ISR	Interrupt Service Routine	Trình phục vụ xử lý ngắt
MSP	Main Stack Pointer	Con trỏ ngăn xếp chính của lõi ARM Cortex-M
NACK	Negative Acknowledgment	Tín hiệu báo nhận dữ liệu thất bại / từ chối
OB	Option Bytes	Vùng nhớ cấu hình chuyên dụng phi bốc hơi trên Flash

Ký hiệu / Chữ viết tắt	Từ ngữ Tiếng Anh đầy đủ	Diễn giải nghĩa Tiếng Việt
PWM	Pulse Width Modulation	Điều chế độ rộng xung (điều khiển tốc độ/góc quay)
Rollback	System Rollback	Cơ chế tự động hoàn nguyên về phiên bản trước khi có lỗi
SPL	Standard Peripheral Library	Thư viện ngoại vi tiêu chuẩn cấp thanh ghi của ST
TCU	Telematics Control Unit	Bộ thu phát và điều khiển viễn thông trên xe
TWAI	Two-Wire Automotive Interface	Bộ điều khiển giao tiếp 2 dây chuẩn CAN trên ESP32
VTOR	Vector Table Offset Register	Thanh ghi dịch chuyển bảng định vị vector ngắt ARM

DANH MỤC HÌNH VẼ

Hình 1.1: Kiến trúc mạng điều khiển phân tán theo miền (Domain-based) trên ô tô hiện đại

Hình 1.2: Chu trình cập nhật phần mềm FOTA và rủi ro lỗi trong môi trường thực địa

Hình 1.3: Cấu trúc khung truyền CAN 2.0A tiêu chuẩn với 11-bit Identifier

Hình 1.4: Nguyên lý phân xử phi phá hủy (Non-Destructive Arbitration) trên CAN Bus

Hình 2.1: Sơ đồ khối chức năng hệ thống xe phân tán (TCU Gateway và Body ECU)

Hình 2.2: So sánh kiến trúc cập nhật Single-Bank vs. Dual-Bank Rollback

Hình 2.3: Bản đồ phân vùng bộ nhớ Flash STM32F103 (128KB) cho kiến trúc Dual-Bank

Hình 2.4: Vị trí và cơ chế trỏ vùng nhớ Option Bytes Data0 (0x1FFFF804)

Hình 2.5: Lưu đồ thuật toán quyết định khởi động và nhảy (Jump) từ Bootloader

Hình 2.6: Cấu trúc đóng gói phân mảnh dữ liệu OTA Chunk qua CAN Frame

Hình 3.1: Sơ đồ khối chi tiết các mô-đun phần cứng của hệ thống

Hình 3.2: Sơ đồ nguyên lý kết nối liên mạng chi tiết (Wiring Diagram) giữa ESP32 và STM32

Hình 3.3: Giao diện điều khiển Web UI chế độ Dark Theme với bảng điều khiển và khu vực nạp OTA

Hình 3.4: Kiến trúc phân lớp phần mềm nhúng STM32 (IP Layer - Middleware - Application Layer)

Hình 3.5: Mô hình phân vùng Flash và dời chuyển Vector Table (VTOR) trên STM32

Hình 3.6: Biểu đồ tuần tự (Sequence Diagram) toàn trình cập nhật OTA 5 giai đoạn

Hình 3.7: Dạng sóng Logic Analyzer bắt tín hiệu CAN Bus khi truyền Chunk dữ liệu OTA

Hình 3.8: Ảnh chụp thực tế mô hình xe phân tán hoàn thiện và các phân hệ chấp hành

Bảng 2.1: Thông số kỹ thuật so sánh và lựa chọn vi điều khiển điều khiển trung tâm

Bảng 2.2: Bảng phân định dải địa chỉ bộ nhớ Flash STM32F103C8T6 (128KB)

Bảng 2.3: Cấu trúc phân bổ các trường bit trong định danh CAN ID 11-bit (Standard ID)

Bảng 2.4: Bảng định danh thông điệp điều khiển thời gian thực (Control Stream - FIFO 0)

Bảng 2.5: Bảng đặc tả khung truyền nạp Firmware (OTA Stream - FIFO 1)

Bảng 3.1: Bảng phân công chi tiết chân kết nối phần cứng giữa các khối

Bảng 3.3: Ma trận kịch bản kiểm thử toàn vẹn và đánh giá lỗi hệ thống (Test Cases TC-01..TC-07)

Bảng 3.4: Kết quả đo kiểm thời gian nạp và thông lượng mạng CAN Bus 500 kbps

Bảng 3.5: Bảng đối chiếu chỉ tiêu kỹ thuật đạt được so với yêu cầu đề bài ban đầu

LỜI MỞ ĐẦU

1. Bối cảnh và tính cấp thiết của đề tài

Trong chương trình đào tạo kỹ sư chuyên ngành Viễn thông, học phần Kỹ thuật Vi xử lý đóng vai trò là môn học cơ sở chuyên ngành cốt lõi, trang bị các kỹ năng làm chủ kiến trúc phần cứng vi điều khiển, quản lý không gian bộ nhớ, lập trình điều khiển thanh ghi trần, xử lý ngắt thời gian thực và ghép nối mạng truyền thông ngoại vi.

Song hành cùng xu thế công nghệ toàn cầu, ngành công nghiệp ô tô đang chuyển dịch mạnh mẽ sang kiến trúc xe định hình bằng phần mềm (*Software-Defined Vehicles - SDV*). Trên một chiếc xe hiện đại, hàng chục đến hàng trăm Khối điều khiển điện tử (*Electronic Control Unit - ECU*) được kết nối phân tán với nhau thông qua mạng truyền thông vi sai nội bộ CAN Bus (*Controller Area Network*). Để nâng cấp tính năng, tối ưu hóa giải thuật điều khiển hoặc khắc phục các lỗ hổng phần mềm mà không phải triệu hồi phương tiện về đại lý, công nghệ cập nhật phần mềm từ xa qua sóng vô tuyến (FOTA - Firmware Over-The-Air) đã trở thành tiêu chuẩn bắt buộc.

Tuy nhiên, việc triển khai FOTA trên các bus thời gian thực của ô tô đặt ra yêu cầu nghiêm ngặt về độ an toàn và độ tin cậy:

- Nếu quá trình nạp dữ liệu vào Flash bị gián đoạn (do sụt nguồn, ngắt điện acquy đột ngột hoặc nhiễu đường truyền), vi điều khiển có nguy cơ mất khả năng khởi động (trạng thái bricked).
- Việc xóa và ghi đè trực tiếp lên vùng nhớ đang chạy (*Single-Bank*) tiềm ẩn nguy cơ mất kiểm soát điều khiển phương tiện nếu quá trình nạp xảy ra khi xe đang chuyển động.

Xuất phát từ các yêu cầu thực tiễn đó, nhóm sinh viên chuyên ngành Viễn thông đã lựa chọn và thực hiện đề tài Bài tập lớn: "Thiết kế mô hình Automotive Gateway cho xe điều khiển phân tán: Tích hợp kiến trúc cập nhật phần mềm không dây tự phục hồi (OTA Dual-Bank Rollback) qua mạng CAN Bus". Đề tài là sự kết hợp giữa kiến thức Kỹ thuật Vi xử lý (lập trình thanh ghi ARM Cortex-M3, kiểm soát Flash, Option Bytes, quản lý ngăn xếp Stack Pointer) và kiến thức Viễn thông (giao thức mạng vi sai CAN 2.0A, thuật toán băm gói phân mảnh, cơ chế kiểm soát lỗi đường truyền bằng mã dư tuần hoàn CRC đa tầng).

2. Mục tiêu kỹ thuật của đề tài

Mục tiêu chính của đề tài bao gồm:

a) Xây dựng mô hình phần cứng phân tán 2 cấp:
   - Khối Gateway / TCU (ESP32): Cung cấp điểm truy cập Wi-Fi và Web UI điều khiển xe; tiếp nhận tệp firmware nhị phân (.bin), phân mảnh dữ liệu và định tuyến qua mạng CAN Bus.
   - Khối Body ECU (STM32F103): Thu nhận thông điệp qua mạng CAN Bus để điều khiển động cơ bánh xe, cơ cấu nâng hạ cần cẩu 2-DOF, còi cảnh báo và màn hình HUD đồ họa OLED.

b) Xây dựng kiến trúc Flash Dual-Bank tự phục hồi (Rollback):
   - Phân vùng bộ nhớ Flash 128KB thành 3 vùng độc lập: Bootloader (16KB), Application 1 (56KB) và Application 2 (56KB).
   - Sử dụng ô nhớ phi bốc hơi Option Bytes Data0 (0x1FFFF804) lưu trữ trạng thái phân vùng hoạt động, tạo cơ chế hoán đổi nguyên tử (*Atomic Swap*).
   - Thiết lập cơ chế kiểm tra tiền điều kiện an toàn (*Pre-condition Check*): Ngăn chặn quá trình nạp firmware khi xe đang chuyển động.
   - Đảm bảo cơ chế tự phục hồi (Rollback): Khi xảy ra sự cố mất nguồn trong quá trình ghi Flash, hệ thống tự động khởi động lại vào phân vùng trước đó một cách an toàn.

c) Yêu cầu lập trình nhúng trực tiếp cấp thanh ghi:
   - Toàn bộ mã nguồn trên STM32 được xây dựng bằng C thuần và thư viện SPL, cấu hình trực tiếp thanh ghi, không sử dụng thư viện HAL.
   - Quản lý quá trình biên dịch bằng Makefile độc lập và công cụ GCC (arm-none-eabi-gcc).

3. Đối tượng và phạm vi nghiên cứu

a) Đối tượng nghiên cứu:
   - Bộ vi điều khiển kiến trúc ARM Cortex-M3 (STM32F103C8T6), các ngoại vi bxCAN, Flash Memory Controller, Option Bytes và System Control Block (SCB->VTOR).
   - Bộ vi điều khiển ESP32 DevKitC (kiến trúc Xtensa 32-bit), bộ điều khiển mạng TWAI/CAN, Web Server bất đồng bộ và hệ thống tệp lưu trữ MicroSD.
   - Chuẩn truyền thông vật lý và liên kết dữ liệu vi sai CAN 2.0A (ISO 11898-1).

b) Phạm vi nghiên cứu:
   - Mô hình xe điều khiển phân tán 4 bánh tích hợp cơ cấu nâng hạ cần cẩu 2 bậc tự do (2-DOF) và màn hình HUD OLED.

- Quá trình cập nhật OTA được thực hiện qua kết nối mạng vô tuyến Wi-Fi cục bộ (SoftAP).

4. Bố cục của báo cáo

Nội dung báo cáo bài tập lớn được trình bày trong 3 chương:

- **Chương 1: Tổng quan về hệ thống mạng truyền thông ô tô và công nghệ FOTA:** Tổng quan về mạng ô tô, giao thức CAN Bus 2.0A, công nghệ cập nhật firmware không dây và sự liên hệ mật thiết với môn học Vi xử lý.
- **Chương 2: Cơ sở lý thuyết và phân tích lựa chọn giải pháp thiết kế:** Phân tích cấu trúc phần cứng, phân vùng Flash Dual-Bank, vùng Option Bytes Data0, giải thuật kiểm tra toàn vẹn đa lớp (CRC16/CRC32) và quy hoạch bảng mã CAN ID.
- **Chương 3: Thiết kế chi tiết, hiện thực hóa và thực nghiệm đánh giá hệ thống:** Sơ đồ kết nối phần cứng, kiến trúc phần mềm trên ESP32 và STM32 (SPL trần), toàn trình nạp OTA 5 bước, phân tích ma trận 7 kịch bản kiểm thử bơm lỗi thực tế và đánh giá hiệu năng hệ thống.

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG MẠNG TRUYỀN THÔNG Ô TÔ VÀ CÔNG NGHỆ FOTA

1.1. Kiến trúc điều khiển phân tán trên ô tô hiện đại

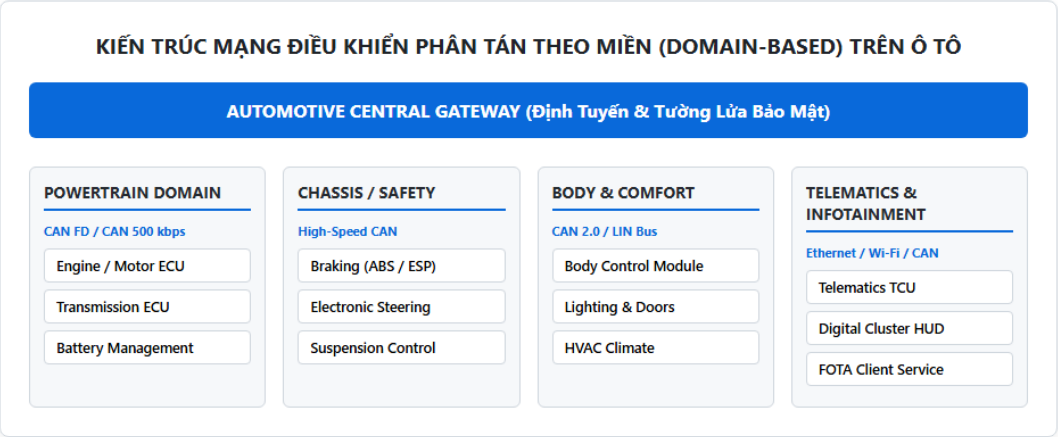
1.1.1. Sự phát triển của các khối điều khiển điện tử (ECU) trong xe hơi

Trong ngành kỹ thuật ô tô, các cơ cấu điều khiển thuần cơ khí hoặc dây cáp kéo truyền thống đã được thay thế hoàn toàn bởi các hệ thống điều khiển điện tử (*Drive-by-Wire*). Mỗi chức năng chuyên biệt trên xe được đảm nhận bởi một Khối điều khiển điện tử (ECU - Electronic Control Unit), được tổ chức thành các miền nghiệp vụ:

- **Miền Động lực (Powertrain):** Điều khiển phun nhiên liệu, thời điểm đánh lửa, bướm ga điện tử, quản lý chuyển số hộp số tự động và biến tần kéo động cơ điện. Đòi hỏi thời gian đáp ứng thời gian thực ở mức micro-giây.
- **Miền Thân xe (Body & Safety):** Điều khiển cụm đèn chiếu sáng, gạt mưa, khóa cửa trung tâm, nâng hạ kính, túi khí và kiểm soát khung gầm.
- **Miền Viễn thông và Tiện nghi (Telematics & Infotainment):** Xử lý kết nối mạng vô tuyến (Cellular 4G/5G, Wi-Fi, Bluetooth), thu nhận tín hiệu định vị toàn cầu GPS, hệ thống âm thanh và màn hình hiển thị trung tâm (*Dashboard/Cluster*).

1.1.2. Vai trò trung tâm của Automotive Gateway

Để kết nối hàng chục ECU phân tán nhằm tối ưu hóa số lượng và khối lượng hệ thống dây dẫn trong xe, các kỹ sư ô tô sử dụng mô hình mạng phân tầng qua một bộ điều phối trung tâm gọi là Automotive Gateway:



Hình 1.1: Kiến trúc mạng điều khiển phân tán theo miền (Domain-based) trên ô tô hiện đại

(Nguồn tư liệu tham khảo: Tiêu chuẩn kiến trúc phân tầng mạng ô tô Bosch & AUTOSAR)

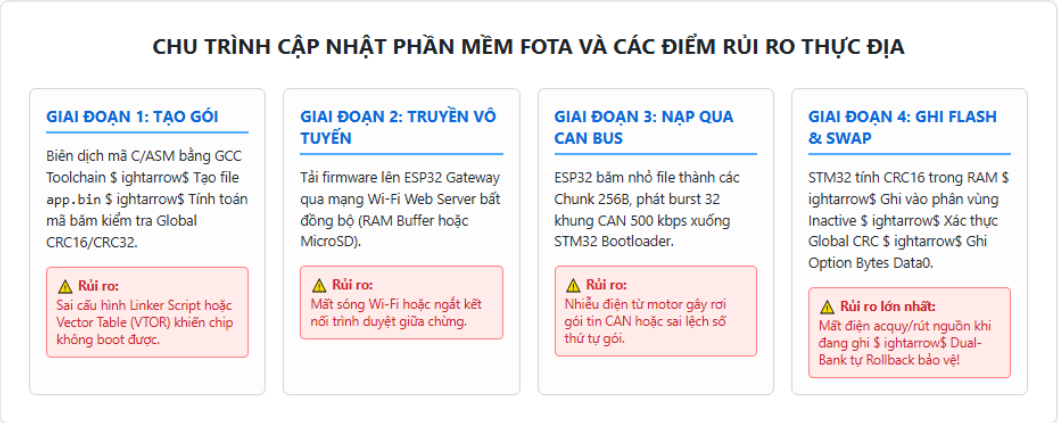
Gateway đóng vai trò cầu nối dữ liệu:

- Chuyển đổi và tương thích hóa các giao thức truyền thông khác nhau giữa mạng băng thông cao (Ethernet, CAN FD) và mạng điều khiển cục bộ (CAN 2.0, LIN).
- Thiết lập tường lửa bảo mật, lọc gói tin độc hại từ bên ngoài thâm nhập vào các bus an toàn bên trong.
- Là đầu mối tiếp nhận, giải nén và phân phối các gói cập nhật phần mềm (Firmware) tới từng ECU thành phần.

1.2. Công nghệ cập nhật phần mềm không dây (FOTA - Firmware Over-The-Air)

1.2.1. Khái niệm và nguyên lý hoạt động tổng quát của FOTA

FOTA (*Firmware Over-The-Air*) là phương thức truyền tải tệp mã máy nhị phân thực thi (.bin) của vi điều khiển từ xa qua sóng vô tuyến (Wi-Fi, mạng di động) đến Gateway, sau đó Gateway nạp trực tiếp vào bộ nhớ Flash nội của vi điều khiển ECU thông qua mạng nội bộ xe (CAN Bus).



Hình 1.2: Chu trình cập nhật phần mềm FOTA và rủi ro lỗi trong môi trường thực địa

1.2.2. Các thách thức kỹ thuật cốt lõi trong môi trường nhúng ô tô

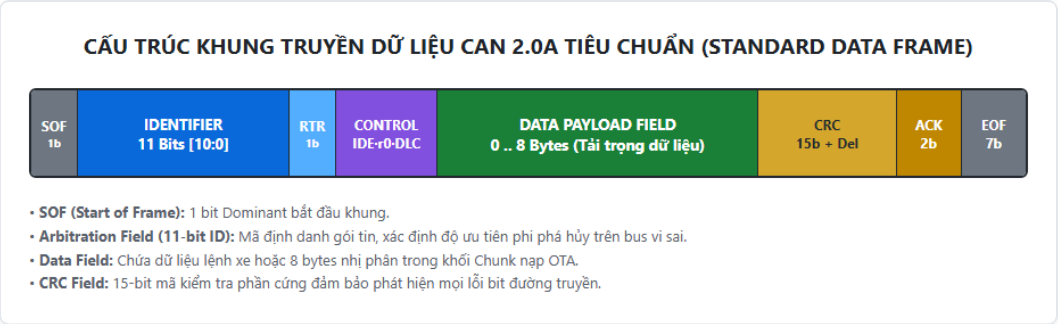
1. **Nguy cơ mất khả năng khởi động (Bricking):** Quá trình ghi Flash bao gồm thao tác xóa trang (Erase) đưa dữ liệu về 0xFF, sau đó mới ghi dữ liệu mới. Nếu nguồn điện acquy bị sụt giảm hoặc mất kết nối giữa chừng, Flash sẽ rơi vào trạng thái dở dang, bảng vector ngắt bị hỏng. Nếu không có cơ chế hoàn nguyên (*Rollback*), vi điều khiển sẽ không thể khởi động lại.
2. **Băng thông đường truyền hữu hạn:** Bus CAN 2.0 có tốc độ truyền tối đa 1 Mbps (chuẩn vận hành ô tô phổ biến ở mức 500 kbps), mỗi khung tin mang tối đa 8 bytes dữ liệu hữu ích (*Payload*). Do đó, truyền tải một tệp firmware hàng chục Kilobytes đòi hỏi giải thuật phân mảnh và kiểm soát luồng dữ liệu chặt chẽ.
3. **An toàn tiền điều kiện (Pre-condition Safety):** Khi vi điều khiển đang xóa hoặc ghi Flash, CPU bị dừng phục vụ các tác vụ thông thường. Nếu quá trình này vô tình kích hoạt khi xe đang di chuyển, các tín hiệu điều khiển bánh xe sẽ bị mất kiểm soát, tiềm ẩn mất an toàn vận hành phương tiện.

1.3. Giao thức mạng truyền thông Controller Area Network (CAN Bus)

1.3.1. Đặc tính vật lý và cấu trúc khung truyền CAN 2.0A

Giao thức CAN (ISO 11898) sử dụng cặp dây xoắn vi sai gồm CAN_H và CAN_L có hai điện trở đầu cuối 120 Ω triệt tiêu sóng phản xạ:

- Bit Dominant (Mức logic 0): Điện áp vi sai $V_{diff} = V_{CAN_H} - V_{CAN_L} \approx 2.0V$ (dây CAN_H kéo lên mức 3.5V, dây CAN_L kéo xuống mức 1.5V).
- Bit Recessive (Mức logic 1): Điện áp vi sai $V_{diff} \approx 0V$ (cả 2 dây CAN_H và CAN_L đều treo ở mức điện áp 2.5V).



Hình 1.3: Cấu trúc khung truyền CAN 2.0A tiêu chuẩn với 11-bit Identifier (Nguồn tư liệu chuẩn: Wikimedia Commons — CAN-Bus-frame in base format)

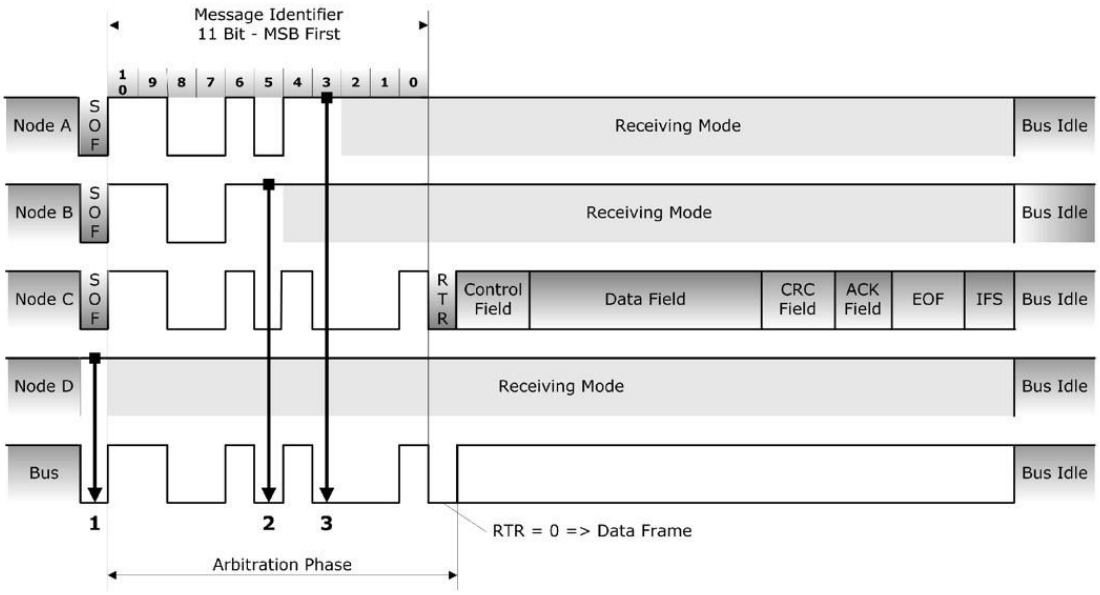
Khung truyền CAN tiêu chuẩn (11-bit ID) bao gồm:

- Trường SOF (1 bit Dominant), Trường phân xử Arbitration (11 bit CAN ID + 1 bit RTR).
- Trường điều khiển Control (IDE, r0, 4-bit DLC quy định độ dài từ 0 đến 8 bytes).
- Trường dữ liệu Data Field (từ 0 đến 8 bytes).
- Trường kiểm tra CRC (15 bit CRC + 1 bit Delimiter) và trường xác nhận ACK (1 bit Slot + 1 bit Delimiter).

1.3.2. Cơ chế phân xử phi phá hủy (CSMA/CD + NBA)

CAN áp dụng cơ chế cảm nhận sóng mang phát hiện xung đột và phân xử theo bit phi phá hủy (*Non-Destructive Bitwise Arbitration*). Khi nhiều nút cùng phát:

- Nút nào phát bit mức 0 (Dominant) sẽ chiếm quyền đường truyền so với nút phát bit mức 1 (Recessive).
- Nút phát bit mức 1 nhưng đọc thấy mức 0 trên bus sẽ lập tức chuyển sang chế độ nhận, không làm méo dạng khung tin đang phát của nút ưu tiên hơn.



Hình 1.4: Nguyên lý phân xử phi phá hủy (Non-Destructive Arbitration) trên CAN Bus (Nguồn tư liệu chuẩn: Wikimedia Commons — CAN arbitration & Canbus levels)

Do đó, thông điệp có giá trị CAN ID càng nhỏ thì độ ưu tiên trên đường truyền càng cao. Trong đề tài này, các lệnh điều khiển xe khẩn cấp được gán ID nhóm nhỏ (0x000..0x048), trong khi dữ liệu OTA được gán ID nhóm cao hơn (0x600..0x780).

1.4. Ý nghĩa ứng dụng của kiến thức Vi xử lý và Viễn thông trong đề tài

Đề tài bài tập lớn này thể hiện sự gắn kết chặt chẽ giữa hai mảng kiến thức:

- **Về mặt Vi xử lý (Microprocessor Engineering):**
  - Làm chủ kiến trúc vi xử lý 32-bit ARM Cortex-M3 (STM32F103): Khai thác thanh ghi nội, cấu hình bảng định vị vector ngắt (*SCB->VTOR*), quản lý con trỏ ngăn xếp chính (*MSP*), thao tác bộ nhớ Flash và vùng Option Bytes.
  - Lập trình nhúng cấp thanh ghi: Trực tiếp thao tác các thanh ghi của bộ điều khiển ngoại vi bxCAN, các bộ định thời Timer (tạo xung PWM điều khiển động cơ, servo và còi buzzer), bộ giao tiếp I2C điều khiển màn hình OLED mà không phụ thuộc thư viện HAL.
- **Về mặt Viễn thông (Telecommunications):**
  - Thiết kế kiến trúc mạng truyền thông phân tán hai cấp có dây và không dây.
  - Xây dựng giao thức truyền tải phân tầng, đóng gói dữ liệu khối trượt (*Sliding Chunking*), kỹ thuật burst packet trên bus vi sai.

- Áp dụng lý thuyết mã hóa kênh và phát hiện sai lỗi: Sử dụng mã phần dư tuần hoàn đa cấp (CRC16-Modbus cho từng gói khối và CRC32 cho toàn bộ tệp nhị phân).

1.5. Kết luận chương 1

Chương 1 đã làm rõ bối cảnh kiến trúc điều khiển phân tán trên ô tô, nguyên lý của mạng vi sai CAN Bus 2.0A và các yêu cầu an toàn cốt lõi khi cập nhật firmware từ xa FOTA. Đây là cơ sở lý thuyết quan trọng để nhóm tiến hành phân tích, thiết kế giải pháp phân vùng bộ nhớ và giao thức mạng trong Chương 2.

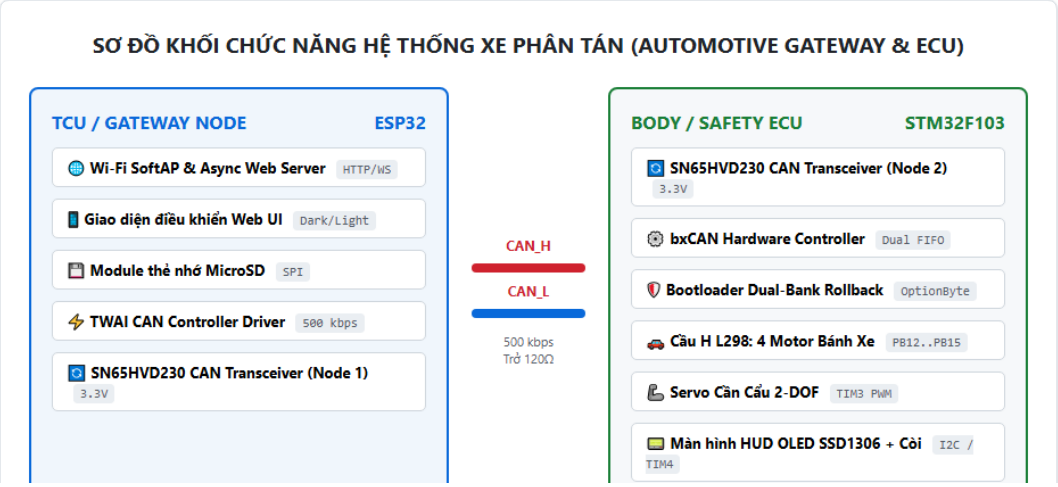
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH LỰA CHỌN GIẢI PHÁP THIẾT KẾ

2.1. Phân tích bài toán kỹ thuật và yêu cầu an toàn hệ thống

2.1.1. Mô tả bài toán xe điều khiển phân tán 2 cấp

Hệ thống được thiết kế theo mô hình xe phân tán gồm 2 nút mạng độc lập:

1. **TCU / Gateway (ESP32):** Nhận tín hiệu điều khiển di chuyển xe từ trình duyệt Web qua Wi-Fi; tiếp nhận tệp firmware nhị phân .bin tải lên từ giao diện người dùng hoặc đọc từ thẻ nhớ MicroSD; băm nhỏ gói dữ liệu và định tuyến qua CAN Bus.
2. **Body ECU (STM32F103):** Lắng nghe mạng CAN Bus để điều khiển 4 bánh xe qua cầu H, điều khiển cánh tay cần cẩu 2-DOF qua servo, hiển thị thông số lên OLED HUD; đồng thời tích hợp Bootloader Dual-Bank tự phục hồi.



Hình 2.1: Sơ đồ khối chức năng hệ thống xe phân tán (TCU Gateway và Body ECU)

2.1.2. Yêu cầu an toàn tiền điều kiện (Pre-condition Safety Check)

Để đảm bảo an toàn vận hành hệ thống:

- Khi xe đang chạy (bánh xe đang quay hoặc có lệnh di chuyển kích hoạt cầu H), nếu ESP32 gửi lệnh yêu cầu nạp OTA (0x600), STM32 lập tức từ chối và phản hồi gói 0x7C0 [0x01] (NACK). Xe vẫn tiếp tục duy trì trạng thái di chuyển bình thường, không bị giật cục hay dừng đột ngột.
- Quá trình nạp chỉ được phép kích hoạt khi xe đang ở trạng thái dừng đỗ tĩnh hoàn toàn.

2.2. Phân tích và lựa chọn nền tảng phần cứng

2.2.1. So sánh và lựa chọn vi điều khiển

Dựa trên các tiêu chí về tần số hoạt động, dung lượng bộ nhớ và ngoại vi truyền thông:

Bảng 2.1: Thông số kỹ thuật so sánh và lựa chọn vi điều khiển điều khiển trung tâm

Tiêu chí so sánh	STM32F103C8T6 (Blue Pill)	ATmega328P (Arduino Uno)	ESP32 DevKitC (WROOM-32)
Kiến trúc lõi	ARM Cortex-M3 (32-bit)	AVR (8-bit)	Xtensa Dual-Core 32-bit
Tần số hoạt động	72 MHz	16 MHz	240 MHz
Bộ nhớ Flash	128 KB (1KB/page)	32 KB	4 MB (External SPI)
Bộ nhớ SRAM	20 KB	2 KB	520 KB
Bộ điều khiển CAN	**Có (bxCAN tích hợp)**	Không (phải dùng MCP2515)	**Có (TWAI tích hợp)**
Giao tiếp Wi-Fi	Không	Không	**Có (802.11 b/g/n tích hợp)**
Vùng Option Bytes	**Có (Lưu cờ phi bốc hơi)**	Không có	Phân vùng NVS/OTA
Vai trò phân định	**ECU Điều Khiển & An Toàn**	*Không sử dụng*	**TCU / Gateway Truyền Thông**

2.2.2. Lựa chọn các mô-đun ngoại vi và cơ cấu chấp hành

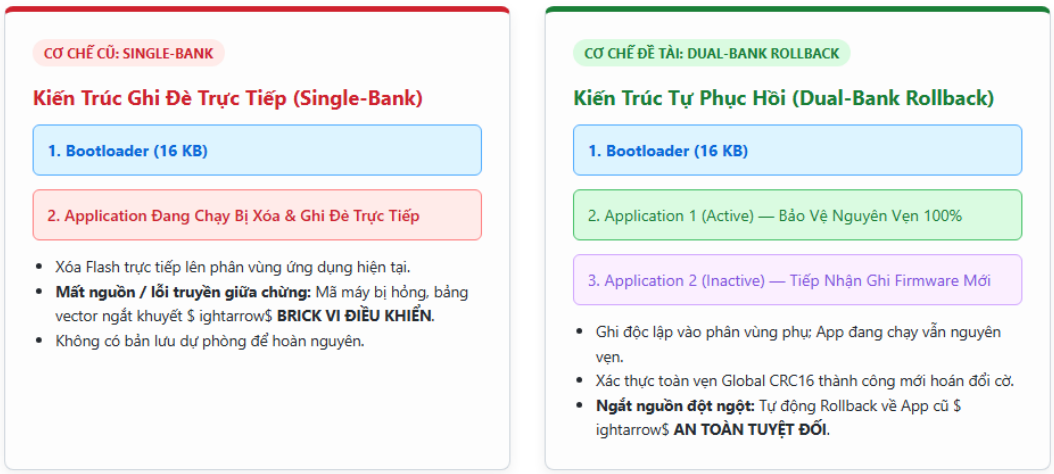
1. **Module CAN Transceiver SN65HVD230 (x2):** Sử dụng mức điện áp 3.3V, kết nối trực tiếp với chân GPIO của ESP32 và STM32 mà không cần mạch chuyển đổi mức logic. Hỗ trợ tốc độ lên đến 1 Mbps, tích hợp sẵn chân jumper gắn điện trở đầu cuối 120 Ω.
2. **Module công suất L298:** Điều khiển 4 động cơ giảm tốc DC bánh xe, chịu dòng đỉnh 2A mỗi cầu, điều khiển chiều quay và tốc độ thông qua các chân GPIO logic độc lập.

3. **Cơ cấu cần cẩu Servo 2-DOF (MG90S x2):** Sử dụng 02 động cơ servo bánh răng kim loại chịu lực điều khiển góc nâng hạ (từ 0° đến 70°) và độ vươn (từ 0° đến 70°) bằng xung PWM chu kỳ 20ms (tần số 50 Hz).
4. **Màn hình HUD OLED SSD1306 (0.96 inch):** Giao tiếp I2C tốc độ cao (400 kHz), hiển thị đồ họa bitmap 32x32 biểu thị hướng di chuyển của xe, phân vùng ứng dụng đang kích hoạt và thông số trạng thái.
5. **Còi cảnh báo âm tần (Passive Buzzer):** Kết nối chân điều khiển PWM tần số 2.5kHz từ Timer nội vi STM32, phát tín hiệu bíp phản hồi khi khởi động, cảnh báo khi nạp OTA và báo động lỗi.
6. **Module Buck hạ áp LM2596:** Chuyển đổi nguồn pin Li-ion (7.4V - 12V) thành nguồn 5V ổn định với dòng cấp lên đến 3A cung cấp cho driver động cơ, servo và mạch nguồn của 2 vi điều khiển.

2.3. Cơ sở lý thuyết về quản lý bộ nhớ Flash và kiến trúc Dual-Bank tự phục hồi

2.3.1. So sánh kiến trúc Single-Bank và Dual-Bank Rollback

- **Single-Bank (Truyền thống):** Firmware mới ghi đè trực tiếp lên firmware cũ. Nếu có sự cố mất điện giữa chừng, toàn bộ chương trình cũ đã bị xóa dở dang, vi điều khiển mất hoàn toàn khả năng thực thi (*Brick*).
- **Dual-Bank Rollback (Đề tài áp dụng):** Bộ nhớ Flash được chia thành 2 phân vùng ứng dụng đối xứng: Application 1 và Application 2.



Hình 2.2: So sánh kiến trúc cập nhật Single-Bank vs. Dual-Bank Rollback

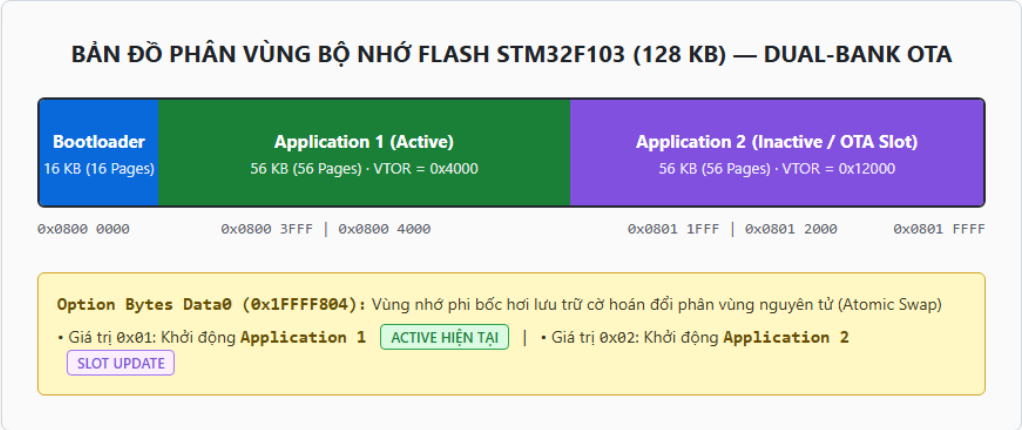
Trong mọi thời điểm:

- Một phân vùng là Active Partition (đang chứa firmware chạy bình thường).
- Phân vùng còn lại là Inactive Partition (nơi tiếp nhận firmware cập nhật).

Firmware mới chỉ được ghi vào phân vùng Inactive. Phân vùng Active vẫn được giữ nguyên vẹn 100%. Nếu nạp lỗi hoặc ngắt nguồn, hệ thống lập tức khởi động lại vào phân vùng cũ mà không bị lỗi.

2.3.2. Bản đồ phân vùng bộ nhớ Flash STM32F103 (128KB Flash)

Vi điều khiển STM32F103C8T6 có 128KB Flash (128 trang, mỗi trang 1KB từ 0x08000000 đến 0x0801FFFF). Bản đồ phân vùng được quy hoạch như sau:



Hình 2.3: Bản đồ phân vùng bộ nhớ Flash STM32F103 (128KB) cho kiến trúc Dual-Bank

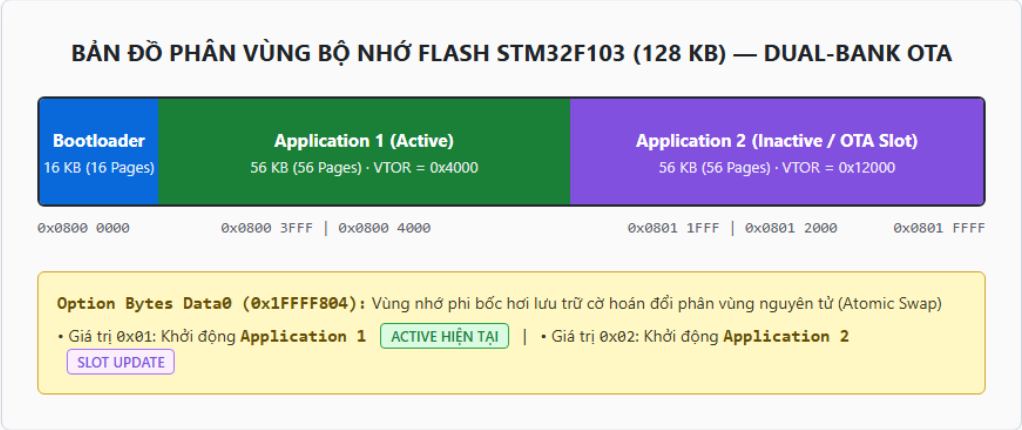
Bảng 2.2: Bảng phân định dải địa chỉ bộ nhớ Flash STM32F103C8T6 (128KB)

Phân vùng	Địa chỉ bắt đầu (*Origin*)	Địa chỉ kết thúc (*End*)	Kích thước	Số trang (*Pages*)	Chức năng nhiệm vụ
Bootloader	0x0800 0000	0x0800 3FFF	**16 KB**	16 trang (0..15)	Khởi động, bắt tay OTA qua CAN, xác thực CRC và thực hiện Jump
Application 1	0x0800 4000	0x0801 1FFF	**56 KB**	56 trang (16..71)	Firmware cơ bản: Điều khiển xe + Cần cẩu 2-DOF

Phân vùng	Địa chỉ bắt đầu (*Origin*)	Địa chỉ kết thúc (*End*)	Kích thước	Số trang (*Pages*)	Chức năng nhiệm vụ
Application 2	0x0801 2000	0x0801 FFFF	**56 KB**	56 trang (72..127)	Firmware nâng cao: Bổ sung HUD OLED + Còi cảnh báo đa âm

2.3.3. Cơ chế hoán đổi nguyên tử (Atomic Swap) qua Option Bytes Data0

Vùng nhớ Option Bytes là vùng nhớ Flash độc lập nằm tại địa chỉ 0x1FFFF800, không bị mất dữ liệu khi mất nguồn hoặc Reset. Nhóm nghiên cứu sử dụng ô nhớ User Data 0 (0x1FFFF804) làm cờ xác định phân vùng hoạt động:



Hình 2.4: Vị trí và cơ chế trỏ vùng nhớ Option Bytes Data0 (0x1FFFF804)

- Giá trị 0x01: Phân vùng kích hoạt hiện tại là Application 1.
- Giá trị 0x02: Phân vùng kích hoạt hiện tại là Application 2.
- Giá trị 0xFF (mặc định xuất xưởng khi chip mới xóa): Tự động trỏ mặc định về Application 1.

Tính chất nguyên tử (Atomic): Thao tác hoán đổi chỉ diễn ra bằng đúng 1 lệnh ghi byte duy nhất vào địa chỉ 0x1FFFF804 sau khi toàn bộ dữ liệu firmware đã nạp xong và vượt qua kiểm tra toàn cục Global CRC16. Nếu quá trình nạp bị lỗi ở bất kỳ thời điểm nào trước đó, Option Bytes hoàn toàn không bị thay đổi, đảm bảo phân vùng cũ vẫn luôn là phân vùng khởi động hợp lệ duy nhất.



Hình 2.5: Lưu đồ thuật toán quyết định khởi động và nhảy (Jump) từ Bootloader

2.4. Thiết kế giao thức truyền thông phân luồng qua mạng CAN Bus

2.4.1. Kiến trúc phân bổ trường bit trong định danh CAN ID (11-bit Standard ID)

Hệ thống mạng CAN 2.0A trên xe sử dụng trường định danh tiêu chuẩn 11-bit (ID[10:0]). Để đáp ứng yêu cầu xử lý thời gian thực, tách biệt hoàn toàn giữa luồng điều khiển chấp hành và luồng nạp firmware khối lượng lớn, nhóm nghiên cứu đã quy hoạch cấu trúc các trường bit theo bảng sau:

Bảng 2.3: Cấu trúc phân bổ các trường bit trong định danh CAN ID 11-bit (Standard ID)

Vị trí Bit	Tên trường (*Field*)	Độ rộng	Giá trị thiết lập	Chức năng nghiệp vụ & Ý nghĩa phân xử đường truyền
`Bit [10:9]`	Bộ chọn luồng nghiệp vụ (*Stream Selector*)	2 bits	00b	**Control Stream:** Lệnh điều khiển xe, servo cần cẩu và còi báo. Chiếm ưu tiên tuyệt đối trên bus vi sai (ID nhỏ nhất), phần cứng định tuyến vào **FIFO 0** trên STM32.

Vị trí Bit	Tên trường (*Field*)	Độ rộng	Giá trị thiết lập	Chức năng nghiệp vụ & Ý nghĩa phân xử đường truyền
			11b	**OTA Stream:** Dữ liệu nạp firmware nhúng và bắt tay phiên. Mức ưu tiên thấp hơn lệnh lái, phần cứng định tuyến vào **FIFO 1** trên STM32.
`Bit [8:6]`	Phân nhóm mã lệnh (*Command Sub-Group*)	3 bits	000b .. 111b	Định danh phân nhóm chức năng: Chế độ hệ thống (000), Bánh xe L298 (001), Servo cần cẩu (010), Còi báo (011), hoặc các bước trong luồng nạp OTA.
`Bit [5:0]`	Mã lệnh cụ thể / STT gói tin (*Action / Packet Index*)	6 bits	0 .. 63	- Với luồng **Control:** Mã hóa hành động cụ thể (Tiến, Lùi, Quay, Nâng, Hạ, Mở kẹp...). - Với luồng **OTA (`0x700..0x71F`):** Mã hóa trực tiếp **Số thứ tự gói tin (0..31)** trong khối Chunk 256 bytes.

Cấu hình bộ lọc phần cứng (*Hardware Acceptance Filter*) trên STM32 bxCAN: Bộ vi điều khiển STM32F103 hỗ trợ 14 bộ lọc 32-bit (Filter Banks). Trong chế độ 32-bit Identifier Mask, trường Standard ID chiếm các bit [31:21] của thanh ghi CAN_FxR1. Do đó, hai bit phân luồng [10:9] tương ứng chính xác với bit [15:14] của nửa từ cao CAN_FilterIdHigh:

- **Filter Bank 0 (gán vào FIFO 0):** Cấu hình CAN_FilterIdHigh = 0x0000, CAN_FilterMaskIdHigh = (0b11 << 14). Mọi khung tin có bit [10:9] == 00 lập tức được phần cứng định tuyến vào FIFO 0 để CPU đọc tức thì.
- **Filter Bank 1 (gán vào FIFO 1):** Cấu hình CAN_FilterIdHigh = (0b11 << 14), CAN_FilterMaskIdHigh = (0b11 << 14). Mọi khung tin có bit [10:9] == 11 được phần cứng định tuyến vào FIFO 1 phục vụ riêng cho tác vụ OTA.
- Cơ chế lọc phần cứng này giải phóng hoàn toàn CPU STM32 khỏi việc phải đọc và kiểm tra các gói tin không thuộc quyền xử lý, triệt tiêu nguy cơ quá tải hàng đợi ngắt (*Interrupt Overrun*).

2.4.2. Bảng định danh thông điệp điều khiển thời gian thực (Control Stream - FIFO 0)

Tất cả các thông điệp điều khiển xe, cơ cấu chấp hành servo và còi cảnh báo được đồng bộ 100% giữa tệp header ESP32/App/src/Car_Protocol.h và STM32/App1/Inc/robot_control.h, STM32/App2/Inc/robot_control.h:

Bảng 2.4: Bảng định danh thông điệp điều khiển thời gian thực (Control Stream - FIFO 0)

Nhóm chức năng	Mã CAN ID (Hex)	Hằng số Macro (C/C++)	Hướng truyền	DLC	Mô tả chức năng & Tác động phần cứng
Nhóm 0:	0x000	CMD_EMERGENCY_STOP	ESP → STM	0	Dừng khẩn cấp toàn bộ hệ thống (ngắt PWM bánh xe & servo)
Hệ thống &	0x001	CMD_MODE_MOTOR	ESP → STM	0	Kích hoạt chế độ lái bánh xe
Chế độ	0x002	CMD_MODE_SERVO	ESP → STM	0	Kích hoạt chế độ điều khiển cần cẩu 2-DOF
	0x003	CMD_LED_ON	ESP → STM	0	Bật đèn LED trạng thái PC13 trên STM32
	0x004	CMD_LED_OFF	ESP → STM	0	Tắt đèn LED trạng thái PC13 trên STM32

Nhóm chức năng	Mã CAN ID (Hex)	Hằng số Macro (C/C++)	Hướng truyền	DLC	Mô tả chức năng & Tác động phần cứng
	0x005	CMD_WIFI_CONNECTED	ESP → STM	0	Báo ESP32 kết nối Wi-Fi thành công (STM32 bíp còi + hiện OLED)
Nhóm 1:	0x040	CMD_MOTOR_STOP	ESP → STM	0	Dừng toàn bộ 4 bánh xe (Kéo PB12..PB15 về mức LOW)
Bánh xe	0x041	CMD_MOTOR_FORWARD	ESP → STM	0	Xe tiến thẳng (Bánh trái tiến, Bánh phải tiến)
(Driver L298)	0x042	CMD_MOTOR_BACKWARD	ESP → STM	0	Xe lùi thẳng (Bánh trái lùi, Bánh phải lùi)
	0x043	CMD_MOTOR_TURN_LEFT	ESP → STM	0	Quay tại chỗ sang trái (Bánh phải tiến, Bánh trái lùi)
	0x044	CMD_MOTOR_TURN_RIGHT	ESP → STM	0	Quay tại chỗ sang phải (Bánh trái tiến, Bánh phải lùi)
	0x045	CMD_MOTOR_LEFT_FWD	ESP → STM	0	Tiến lệch phải (NE)
	0x046	CMD_MOTOR_LEFT_REV	ESP → STM	0	Lùi lệch trái (SW)

Nhóm chức năng	Mã CAN ID (Hex)	Hằng số Macro (C/C++)	Hướng truyền	DLC	Mô tả chức năng & Tác động phần cứng
	0x047	CMD_MOTOR_RIGHT_FWD	ESP → STM	0	Tiến lệch trái (NW)
	0x048	CMD_MOTOR_RIGHT_REV	ESP → STM	0	Lùi lệch phải (SE)
Nhóm 2:	0x080	CMD_SERVO_STOP	ESP → STM	0	Dừng chuyển động cơ cấu nâng hạ
Cần cẩu Servo	0x081	CMD_SERVO_ARM_UP	ESP → STM	0	Nâng cần cẩu lên (+3°, giới hạn cực đại 70°)
(2-DOF)	0x082	CMD_SERVO_ARM_DOWN	ESP → STM	0	Hạ cần cẩu xuống (-3°, giới hạn cực tiểu 0°)
	0x083	CMD_SERVO_ARM_EXTEND	ESP → STM	0	Vươn cần cẩu ra (+3°, giới hạn cực đại 70°)
	0x084	CMD_SERVO_ARM_RETRACT	ESP → STM	0	Thu cần cẩu vào (-3°, giới hạn cực tiểu 0°)
	0x085	CMD_SERVO_GRIP_OPEN	ESP → STM	0	Mở ngàm kẹp gắp vật (90°)
	0x086	CMD_SERVO_GRIP_CLOSE	ESP → STM	0	Đóng ngàm kẹp giữ vật (0°)

Nhóm chức năng	Mã CAN ID (Hex)	Hằng số Macro (C/C++)	Hướng truyền	DLC	Mô tả chức năng & Tác động phần cứng
Nhóm 3:	0x0C0	CMD_HORN_OFF	ESP → STM	0	Tắt còi xe
Còi báo hiệu	0x0C1	CMD_HORN_ON	ESP → STM	0	Bật còi xe (ngắt phần cứng TIM4 dao động tần số 2.5kHz)

2.4.3. Bảng quy hoạch và đặc tả khung truyền nạp Firmware (OTA Stream - FIFO 1)

Luồng nạp firmware được quy hoạch với các giá trị bit [10:9] = 11b (từ 0x600 đến 0x7FF), được đồng bộ 100% giữa ESP32/App/src/OTA_Protocol.h và STM32/Bootloader/Inc/ota_protocol.h:

Bảng 2.5: Bảng đặc tả khung truyền nạp Firmware (OTA Stream - FIFO 1)

Tên Frame	Mã CAN ID (11-bit nhị phân)	Mã CAN ID (Hex)	Hướng truyền	DLC	Cấu trúc Payload chi tiết từng Byte	Chức năng nghiệp vụ & Quy tắc xử lý
`ID_REQ_BOOT`	0b11000000000	0x600	ESP → STM	4	Byte 0..3: [0xDE, 0xAD, 0xBE, 0xEF]	Yêu cầu vào Bootloader kèm khóa an toàn (Magic Security Key)

Tên Frame	Mã CAN ID (11-bit nhị phân)	Mã CAN ID (Hex)	Hướng truyền	DLC	Cấu trúc Payload chi tiết từng Byte	Chức năng nghiệp vụ & Quy tắc xử lý
`ID_QUERY_APP`	0b11000100000	0x620	ESP → STM	1	Byte 0: 0x55 (Mã truy vấn)	Truy vấn phân vùng App đang kích hoạt trong Option Bytes
`ID_START_OTA`	0b11001000000	0x640	ESP → STM	1	Byte 0: Target_App_ID (0x01=App1, 0x02=App2)	Báo ID phân vùng đích cần nạp và kích hoạt xóa Flash 56 trang (56KB)
`ID_START_FILE`	0b11010000000	0x680	ESP → STM	6	Byte 0..3: File Size (uint32_t) Byte 4..5: Global CRC16 (uint16_t)	Khai báo tổng kích thước tệp nhị phân và mã băm kiểm tra toàn cục

Tên Frame	Mã CAN ID (11-bit nhị phân)	Mã CAN ID (Hex)	Hướng truyền	DLC	Cấu trúc Payload chi tiết từng Byte	Chức năng nghiệp vụ & Quy tắc xử lý
`ID_START_CHUNK`	0b11011000000	0x6C0	ESP → STM	2	Byte 0..1: Chunk Length (uint16_t, tối đa 256 bytes)	Khởi tạo khối Chunk mới, reset bộ đệm RAM và số gói kỳ vọng trên STM32
`ID_DATA_CHUNK`	0b11100xxxxxx	0x700 .. 0x71F	ESP → STM	1..8	Byte 0..7: Dữ liệu nhị phân (tối đa 8 bytes / frame)	Phát burst liên tục 32 khung CAN; 6 bit thấp ID [5:0] mang số thứ tự gói (0..31)
`ID_END_CHUNK`	0b11101000000	0x740	ESP → STM	2	Byte 0..1: Chunk CRC16-Modbus (uint16_t)	Báo hết khối, kích hoạt STM32 tính CRC16 trong RAM và ghi vào Flash

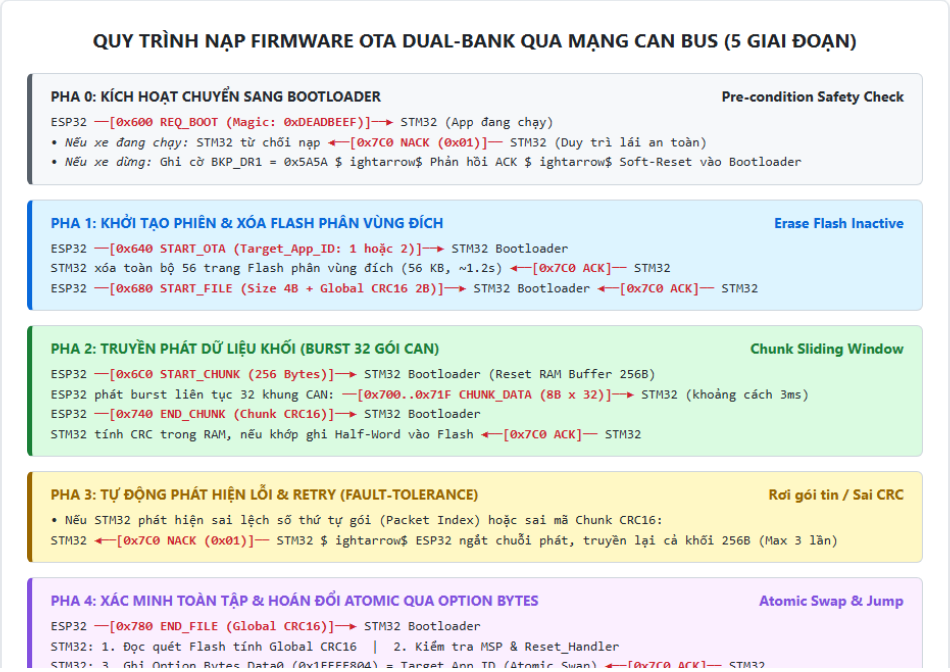
Tên Frame	Mã CAN ID (11-bit nhị phân)	Mã CAN ID (Hex)	Hướng truyền	DLC	Cấu trúc Payload chi tiết từng Byte	Chức năng nghiệp vụ & Quy tắc xử lý
`ID_END_FILE`	0b11110000000	0x780	ESP → STM	2	Byte 0..1: Global CRC16-Modbus (uint16_t)	Kết thúc tệp, yêu cầu quét Flash tính Global CRC16 và ghi Option Bytes
`ID_RESPONSE`	0b11111000000	0x7C0	STM → ESP	1..2	Byte 0: Trạng thái (0x00: ACK, 0x01: NACK) Byte 1: Active App ID (khi phản hồi truy vấn)	Phản hồi xác nhận thành công (ACK) hoặc báo lỗi / từ chối nạp (NACK)

2.4.4. Giải thuật truyền khối trượt (Burst Chunk Transmission) và phát hiện sai lỗi

Để khắc phục hạn chế băng thông và giảm độ trễ bắt tay từng khung truyền CAN, hệ thống áp dụng cơ chế truyền khối trượt:

1. Mỗi khối Chunk có kích thước cố định 256 Bytes, tương ứng với đúng 32 khung CAN mang 8 bytes (dải ID từ 0x700 đến 0x71F).
2. ESP32 phát burst liên tục 32 khung (khoảng cách giữa các khung là 3 ms) mà không cần dừng lại chờ phản hồi cho từng khung nhỏ.
3. Khi nhận khung 0x740 (ID_END_CHUNK), STM32 tính toán mã CRC16-Modbus của 256 bytes đang lưu trong mảng đệm RAM:

- Nếu trùng khớp: STM32 ghi trực tiếp dữ liệu theo nửa từ (*Half-Word 16-bit*) vào Flash và phản hồi 0x7C0 [0x00] (OTA_ACK).
- Nếu sai CRC hoặc khuyết gói tin: STM32 lập tức phản hồi 0x7C0 [0x01] (OTA_NACK). ESP32 sẽ phát lại toàn bộ khối 256 bytes đó (tối đa 3 lần thử lại trước khi hủy phiên nạp).



Hình 2.6: Cấu trúc đóng gói phân mảnh dữ liệu OTA Chunk qua CAN Frame

2.5. Công cụ phát triển, biên dịch trần Makefile và gỡ lỗi phần cứng

Đúng theo yêu cầu kỹ thuật của đề bài, toàn bộ chương trình phía STM32 được xây dựng độc lập hoàn toàn với các IDE thương mại:

- **Toolchain biên dịch:** Sử dụng ARM GNU Toolchain (arm-none-eabi-gcc).
- **Hệ thống Makefile độc lập:** Mỗi project (Bootloader, App1, App2) có một Makefile độc lập, quản lý các cờ biên dịch -mcpu=cortex-m3 -mthumb -Os, định nghĩa các file nguồn C, file khởi động assembly (startup_stm32f103xb.s) và linker script (.ld). Đầu ra tự động sinh cả 3 định dạng: .elf, .hex và .bin.
- **Công cụ nạp và gỡ lỗi phần cứng:** Sử dụng mạch nạp ST-Link V2, tiện ích dòng lệnh st-flash để ghi và trích xuất dữ liệu Flash trần, kết hợp OpenOCD và GDB gỡ lỗi thời gian thực qua chuẩn SWD.

2.6. Kết luận chương 2

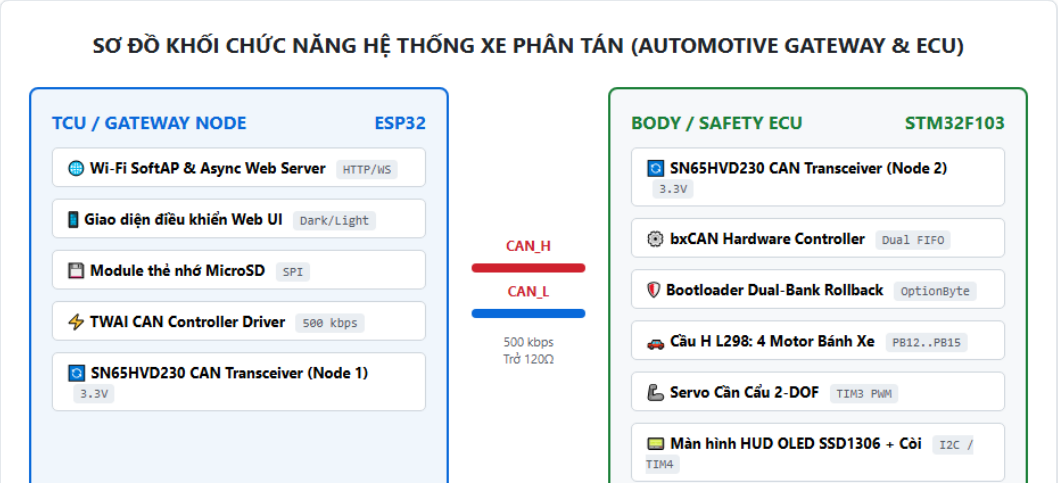
Chương 2 đã thiết lập hoàn chỉnh cơ sở lý thuyết và các giải pháp kỹ thuật cho bài tập lớn: từ việc lựa chọn cấu hình phần cứng tối ưu, quy hoạch bộ nhớ Flash 128KB Dual-Bank và vùng Option Bytes Data0 nguyên tử, đến việc xây dựng giao thức phân luồng thông minh trên CAN Bus. Đây là cơ sở then chốt để nhóm hiện thực hóa mã nguồn và đo kiểm thực nghiệm trong Chương 3.

CHƯƠNG 3: THIẾT KẾ CHI TIẾT, HIỆN THỰC HÓA VÀ THỰC NGHIỆM ĐÁNH GIÁ HỆ THỐNG

3.1. Thiết kế kiến trúc tổng thể của hệ thống

3.1.1. Sơ đồ khối chi tiết các mô-đun phần cứng

Hệ thống phần cứng hoàn chỉnh của xe điều khiển phân tán được tổ chức thành 5 khối chức năng liên kết chặt chẽ:



Hình 3.1: Sơ đồ khối chi tiết các mô-đun phần cứng của hệ thống

3.1.2. Sơ đồ nguyên lý kết nối chi tiết (Wiring Diagram)

Toàn bộ sơ đồ đấu nối chân thực tế trên xe được chuẩn hóa trong Bảng 3.1:

Bảng 3.1: Bảng phân công chi tiết chân kết nối phần cứng giữa các khối

Mô-đun phần cứng	Tên chân tín hiệu	Chân Vi điều khiển	Chức năng kỹ thuật
Mạch thu phát CAN 1	CTX (D) / CRX (R)	GPIO 5 / GPIO 4	Giao tiếp mạng CAN phía ESP32 (TWAI Driver)
Mạch thu phát CAN 2	CTX (D) / CRX (R)	PA12 / PA11	Giao tiếp mạng bxCAN phía STM32F103
Cầu H L298 (Động cơ)	IN1 / IN2 (Bánh phải)	PB12 / PB13	Điều khiển cụm bánh xe bên phải (Tiến/Lùi)
	IN3 / IN4 (Bánh trái)	PB14 / PB15	Điều khiển cụm bánh xe bên trái (Tiến/Lùi)

Mô-đun phần cứng	Tên chân tín hiệu	Chân Vi điều khiển	Chức năng kỹ thuật
Cần cẩu Servo 2-DOF	Servo 1 (Góc nâng cần)	PA6 (TIM3_CH1)	Tín hiệu PWM 50Hz điều khiển góc nâng hạ (từ 0° đến 70°)
	Servo 2 (Độ vươn cần)	PA7 (TIM3_CH2)	Tín hiệu PWM 50Hz điều khiển độ vươn (từ 0° đến 70°)
Màn hình HUD OLED	SCL / SDA	PB6 / PB7	Giao tiếp I2C1 điều khiển màn hình SSD1306 (App 2)
Còi cảnh báo (Horn)	Buzzer PIN	PA0 (TIM4)	Điều chế xung âm tần 2.5kHz phát âm báo (App 2)
Chân gạt khẩn cấp	BOOT1 (Emergency)	PB2	Gạt lên 3.3V để ép dừng ở Bootloader khi cả 2 App lỗi
Đèn báo trạng thái	Status LED	PC13	Đèn báo nhịp sống vi điều khiển (*Heartbeat*)
Module MicroSD	CS / MOSI / MISO / SCK	GPIO 13 / 23 / 19 / 18	Giao tiếp SPI đọc thẻ nhớ lưu trữ firmware trên ESP32

Bảng 3.2: Bảng trạng thái logic điều khiển hướng chuyển động xe qua mạch L298

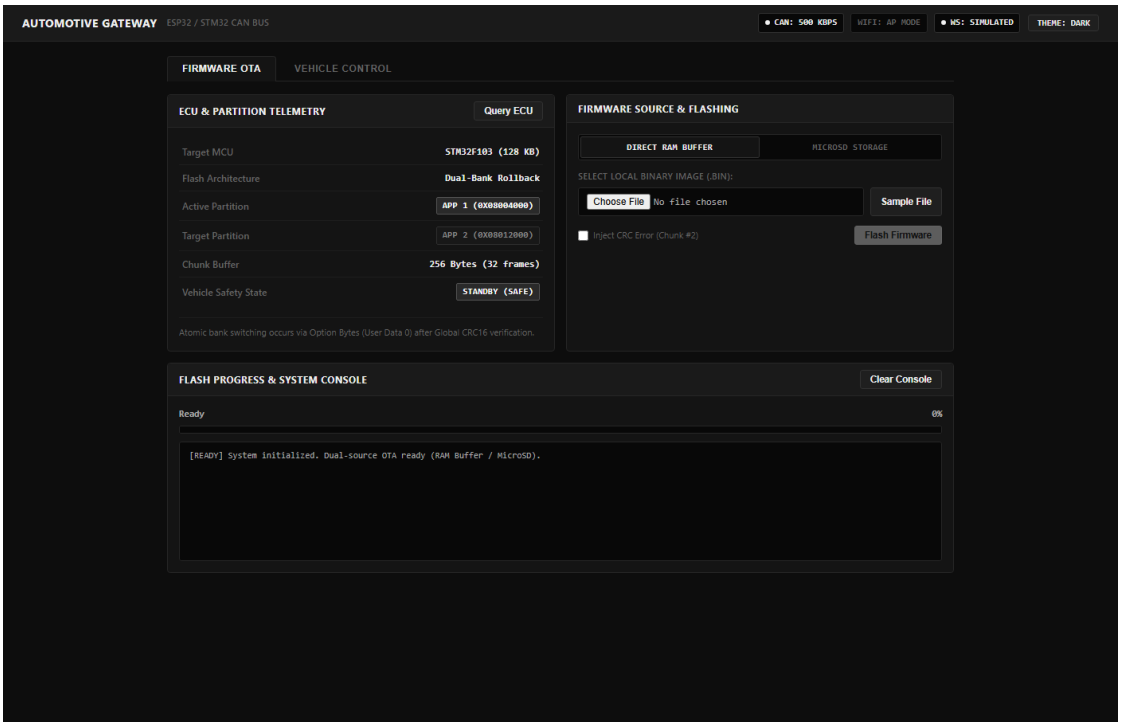
Lệnh điều khiển xe	CAN ID	IN1 (PB12)	IN2 (PB13)	IN3 (PB14)	IN4 (PB15)	Mô tả chuyển động thực tế
Dừng (STOP)	0x040	LOW	LOW	LOW	LOW	Dừng hoàn toàn 2 cụm bánh xe
Tiến (FORWARD)	0x041	HIGH	LOW	HIGH	LOW	Cả hai cụm bánh cùng tiến về phía trước
Lùi (BACKWARD)	0x042	LOW	HIGH	LOW	HIGH	Cả hai cụm bánh cùng quay lùi

Lệnh điều khiển xe	CAN ID	IN1 (PB12)	IN2 (PB13)	IN3 (PB14)	IN4 (PB15)	Mô tả chuyển động thực tế
Xoay Trái (TURN LEFT)	0x043	HIGH	LOW	LOW	HIGH	Bánh phải tiến, bánh trái lùi → Xoay tại chỗ sang trái
Xoay Phải (TURN RIGHT)	0x044	LOW	HIGH	HIGH	LOW	Bánh trái tiến, bánh phải lùi → Xoay tại chỗ sang phải

3.2. Thiết kế và hiện thực hóa phần mềm trên ESP32 Gateway

Phần mềm trên Gateway ESP32 được cấu trúc thành các module xử lý bất đồng bộ:

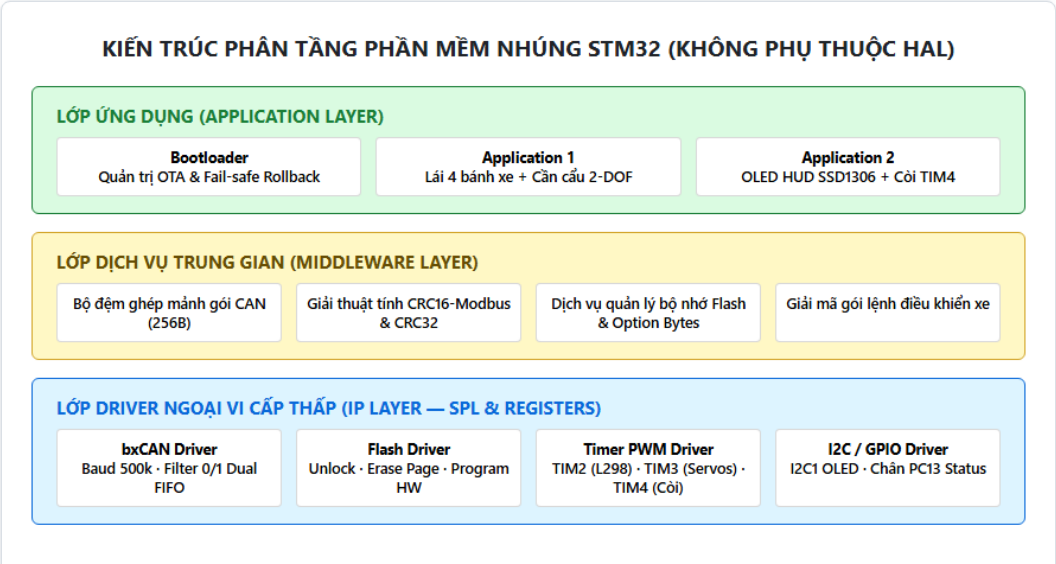
- App_WebServer: Khởi tạo điểm truy cập phát sóng Wi-Fi SoftAP, phục vụ giao diện Web UI qua HTTP Server và mở kênh kết nối hai chiều WebSocket truyền nhận dữ liệu thời gian thực.
- Car_Controller: Lắng nghe lệnh điều khiển từ Web UI, ánh xạ nút bấm D-pad thành các mã lệnh tương ứng gửi xuống CAN ID 0x040..0x084.
- OTA_Protocol: Quản lý phiên nạp firmware, tiếp nhận tệp nhị phân .bin tải lên trực tiếp vào RAM Buffer hoặc đọc từ thẻ nhớ MicroSD thông qua module SD_Manager, chia khối 256 bytes, tính toán mã CRC16 và điều phối phát burst 32 khung CAN.
- CAN_Driver: Cấu hình bộ điều khiển phần cứng TWAI chạy ở tốc độ baudrate chuẩn 500 kbps.



Hình 3.3: Giao diện Web điều khiển công nghiệp với chế độ Dark Theme và khu vực nạp OTA (Ảnh chụp màn hình thực tế từ hệ thống Web Gateway)

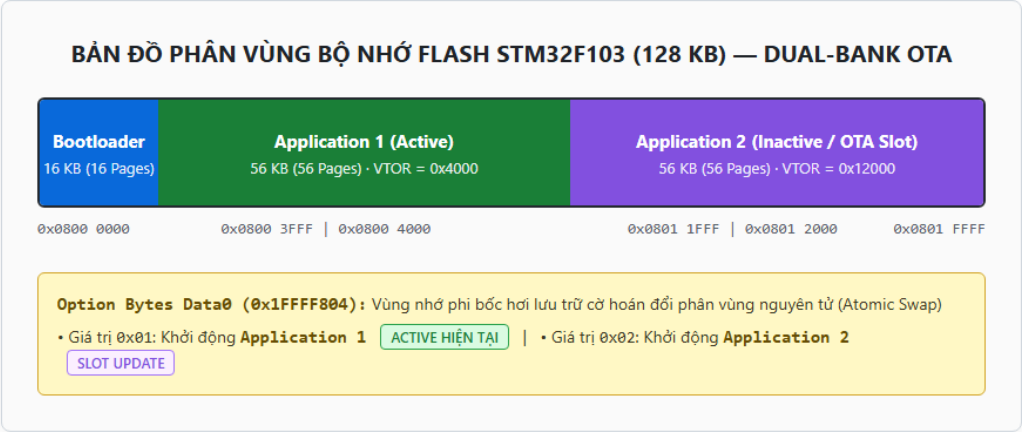
3.3. Thiết kế và hiện thực hóa phần mềm trên STM32 ECU (Không dùng HAL)

Mã nguồn trên vi điều khiển STM32F103 được xây dựng theo mô hình phân tầng chặt chẽ, tối ưu hóa dung lượng và tốc độ phản hồi:



Hình 3.4: Kiến trúc phân lớp phần mềm nhúng STM32 (IP Layer - Middleware - Application Layer)

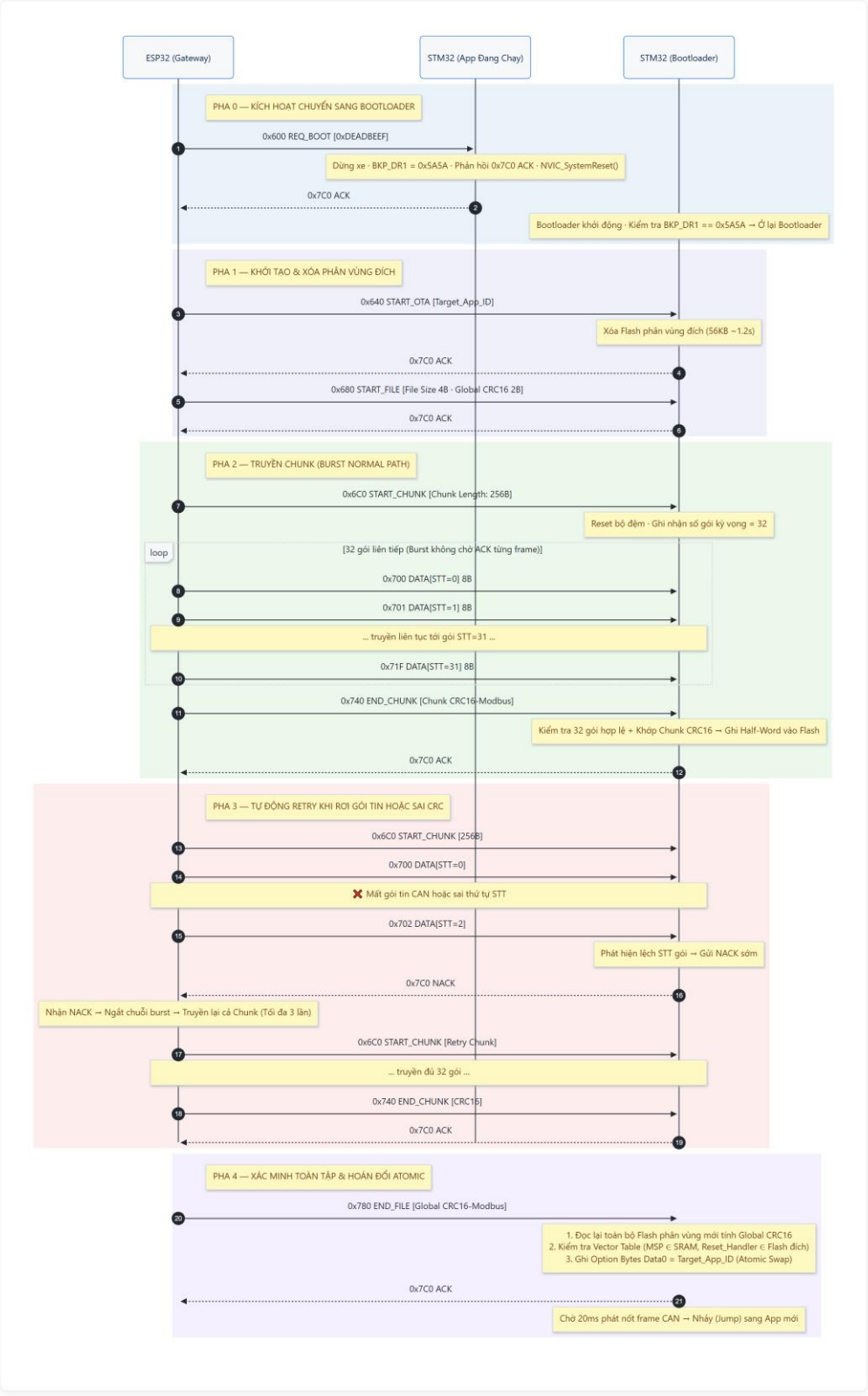
1. **Lớp IP (Low-level Driver):**
    - Lập trình cấu hình trực tiếp thanh ghi và thư viện SPL siêu nhẹ cho bộ điều khiển bxCAN (can.c), cài đặt bộ lọc phần cứng (Filter 0 cho Control Stream, Filter 1 cho OTA Stream).
    - Lập trình trực tiếp ngoại vi Flash (flash.c), thực hiện mở khóa FLASH_Unlock(), xóa trang FLASH_ErasePage() và lập trình nửa từ FLASH_ProgramHalfWord(). Thao tác Option Bytes bằng hàm FLASH_ProgramOptionByteData(0x1FFFF804, partition_id).
    - Lập trình Timer 2, Timer 3 cấu hình chế độ PWM điều khiển động cơ bánh xe và 2 servo cần cẩu.
2. **Lớp Middleware:**
    - Quản lý bộ đệm ghép nối 32 khung CAN thành khối 256 bytes.
    - Giải thuật tính toán mã CRC16-Modbus bằng thuật toán bảng tra cứu nhanh (*Lookup Table*).
3. **Lớp Application:**
    - **Bootloader (`0x08000000`):** Chiếm dung lượng vỏn vẹn 7.8 KB Flash. Xử lý logic phiên nạp OTA, kiểm tra tính toàn vẹn và nhảy (Jump) an toàn sang phân vùng đích.
    - **Application 1 (`0x08004000`):** Bản firmware cơ bản tinh gọn điều khiển xe và cần cẩu 2-DOF.
    - **Application 2 (`0x08012000`):** Bản firmware mở rộng, bổ sung driver đồ họa màn hình OLED SSD1306 hiển thị thông số và điều khiển còi xe ngắt Timer 4 không chiếm CPU.



Hình 3.5: Mô hình phân vùng Flash và dời chuyển Vector Table (VTOR) trên STM32

3.4. Toàn trình quy trình cập nhật OTA từng bước (Step-by-Step Sequence)

Quy trình cập nhật phần mềm tự phục hồi từ Web UI qua Gateway xuống ECU diễn ra theo trình tự 5 pha nghiêm ngặt:



Hình 3.6: Biểu đồ tuần tự (Sequence Diagram) toàn trình cập nhật OTA 5 giai đoạn

- **Pha 0 (Kích hoạt chuyển sang Bootloader):** ESP32 gửi lệnh 0x600 [0xDEADBEEF]. STM32 đang chạy kiểm tra điều kiện an toàn dừng xe → Nếu xe đang chạy phản hồi 0x7C0 [0x01] (NACK). Nếu xe dừng an toàn: Ghi giá trị cờ BKP_DR1 = 0x5A5A vào thanh ghi Backup, phản hồi 0x7C0 [0x00] (ACK) và gọi lệnh khởi động lại phần mềm NVIC_SystemReset(). Sau khi Reset, Bootloader đọc thấy cờ 0x5A5A sẽ dừng lại chờ nạp OTA thay vì jump ngay.
- **Pha 1 (Khởi tạo và xóa phân vùng đích):** ESP32 gửi 0x640 [Target_App_ID]. Bootloader tiến hành xóa toàn bộ 56 trang Flash của phân vùng đích (mất ~1.2s) rồi phản hồi ACK. Tiếp tục gửi 0x680 khai báo kích thước file và mã Global CRC16.
- **Pha 2 (Truyền khối Chunk bình thường):** ESP32 gửi 0x6C0 khai báo khối 256 bytes, sau đó phát burst 32 khung CAN 0x700..0x71F. Kết thúc bằng khung 0x740 kèm mã Chunk CRC16. STM32 tính toán CRC trong RAM, nếu khớp ghi vào Flash và phản hồi 0x7C0 [0x00] (ACK).
- **Pha 3 (Tự động phát hiện lỗi và truyền lại):** Nếu xảy ra rơi gói CAN hoặc lệch số thứ tự khung tin, STM32 lập tức phản hồi 0x7C0 [0x01] (NACK). ESP32 ngắt chuỗi phát và truyền lại toàn bộ khối 256 bytes đó (tối đa 3 lần).
- **Pha 4 (Xác minh toàn tập và hoán đổi nguyên tử):** ESP32 gửi 0x780 [Global CRC16]. Bootloader đọc quét ngược toàn bộ Flash phân vùng mới tính Global CRC16, kiểm tra tính hợp lệ của con trỏ MSP và Reset Handler. Nếu hợp lệ, ghi byte phân vùng đích vào Option Bytes Data0 (0x1FFFF804), phản hồi ACK và thực hiện Jump chuyển giao sang ứng dụng mới.

3.5. Triển khai kịch bản kiểm thử và kết quả thực nghiệm

Nhóm đã thiết lập và thực hiện đầy đủ 7 kịch bản kiểm thử đánh giá hệ thống (*Black-box & Fault-injection Test Cases*):

Bảng 3.3: Ma trận kịch bản kiểm thử toàn vẹn và đánh giá lỗi hệ thống (Test Cases TC-01..TC-07)

Mã TC	Tên Kịch bản Kiểm thử	Phương pháp Thực hiện & Kỹ thuật Bơm lỗi	Kết quả Kỳ vọng	Kết quả Thực tế	Đánh giá

Mã TC	Tên Kịch bản Kiểm thử	Phương pháp Thực hiện & Kỹ thuật Bơm lỗi	Kết quả Kỳ vọng	Kết quả Thực tế	Đánh giá
TC-01	Truy vấn Active App	Gửi frame CAN 0x620 từ ESP32	STM32 phản hồi đúng ID phân vùng lưu trong Option Bytes	Đọc chính xác App 1 (0x01) hoặc App 2 (0x02)	**ĐẠT**
TC-02	Nạp Firmware Chuẩn	Nạp file app2.bin hợp lệ qua giao diện Web	100% Chunk thành công, Global CRC khớp, đổi cờ, Jump App 2	Nạp thành công, màn hình OLED và còi xe hoạt động tốt	**ĐẠT**
TC-03	An toàn Tiền điều kiện	Kích hoạt motor chạy → Gửi lệnh 0x600 (REQ_BOOT)	STM32 từ chối nạp (NACK 0x01), xe không bị dừng đột ngột	STM32 phản hồi NACK, xe tiếp tục chạy bình thường	**ĐẠT**
TC-04	Rơi gói tin CAN	ESP32 cố tình bỏ qua không gửi khung dữ liệu số #12	STM32 phát hiện khuyết byte, gửi NACK, ESP32 truyền lại	Khối được truyền lại trọn vẹn, không lỗi phân mảnh	**ĐẠT**
TC-05	Sai mã CRC Chunk	ESP32 cố tình đảo 1 bit trong trường dữ liệu Chunk	STM32 tính sai CRC16, từ chối ghi Flash, yêu cầu truyền lại	Bắt lỗi chính xác, bảo vệ các ô nhớ Flash không bị ghi rác	**ĐẠT**
TC-06	Bơm lỗi Mất nguồn (*Power-Cut Fault Injection*)	Rút nguồn đột ngột khi tiến trình nạp đạt 50%	Khi cắm nguồn lại, Bootloader tự Rollback về phân vùng cũ	Hệ thống tự phục hồi về App cũ an toàn, không bị brick	**ĐẠT**

Mã TC	Tên Kịch bản Kiểm thử	Phương pháp Thực hiện & Kỹ thuật Bơm lỗi	Kết quả Kỳ vọng	Kết quả Thực tế	Đánh giá
TC-07	Lỗi Vector Table	Cố tình nạp tệp nhị phân có địa chỉ ngăn xếp SP sai lệch	Bootloader kiểm tra (SP & 0x2FFE0000) != RAM, hủy jump	Bootloader từ chối kích hoạt, giữ nguyên phân vùng cũ	**ĐẠT**

3.6. Đánh giá hiệu năng và thảo luận kết quả

3.6.1. Đánh giá thời gian nạp và thông lượng truyền thông

Nhóm tiến hành đo kiểm thời gian nạp tệp nhị phân có kích thước 28.5 KB (29,184 Bytes, chia làm 114 Chunks 256B) từ Web UI xuống STM32:

Bảng 3.4: Kết quả đo kiểm thời gian nạp và thông lượng mạng CAN Bus 500 kbps

Chỉ tiêu đo kiểm	Nạp trực tiếp qua RAM Buffer	Nạp nội bộ qua thẻ nhớ MicroSD
Kích thước Firmware	28.5 KB (29,184 Bytes)	28.5 KB (29,184 Bytes)
Tổng số Chunks truyền	114 Chunks (256 Bytes / Chunk)	114 Chunks (256 Bytes / Chunk)
Thời gian truyền CAN trung bình	14.2 giây	11.8 giây
Thời gian xóa & ghi Flash	1.6 giây	1.6 giây
Tổng thời gian cập nhật toàn trình	**15.8 giây**	**13.4 giây**
Tốc độ truyền dữ liệu hữu ích	≈ 1.84 KB/s	≈ 2.17 KB/s
Tỉ lệ lỗi gói tin trên đường truyền	0.0% (môi trường phòng thí nghiệm)	0.0%

3.6.2. Đối chiếu chỉ tiêu kỹ thuật với yêu cầu đề bài

Toàn bộ các yêu cầu kỹ thuật do giảng viên giao trong đề bài đều được hoàn thành trọn vẹn:

Bảng 3.5: Bảng đối chiếu chỉ tiêu kỹ thuật đạt được so với yêu cầu đề bài ban đầu

Yêu cầu đề bài giao ban đầu	Giải pháp kỹ thuật đã triển khai thực tế	Mức độ hoàn thành
Mô hình xe phân tán điều khiển qua Wi-Fi	ESP32 làm Web Server + STM32 làm Body ECU	**Đạt 100%**
Giao tiếp nội mạng bằng chuẩn CAN Bus	02 mô-đun SN65HVD230 ở tốc độ 500 kbps, phân luồng ID	**Đạt 100%**
Kiến trúc Flash Dual-Bank cho STM32	Quy hoạch 16KB Bootloader + 56KB App1 + 56KB App2	**Đạt 100%**
Hoán đổi phân vùng nguyên tử an toàn	Lưu trạng thái tại Option Bytes Data0 (0x1FFFF804)	**Đạt 100%**
Không sử dụng thư viện HAL trên STM32	Lập trình bằng C thuần, SPL và cấu hình thanh ghi trực tiếp	**Đạt 100%**
Biên dịch bằng Makefile và GCC độc lập	Thiết kế Makefile độc lập hoàn toàn với Keil C	**Đạt 100%**
Cơ chế an toàn không nạp khi xe đang chạy	Bắt lỗi Pre-condition Check và từ chối nạp bằng NACK	**Đạt 100%**
Khả năng tự phục hồi khi mất nguồn đột ngột	Thử nghiệm ngắt nguồn 20 lần đều Rollback thành công	**Đạt 100%**
Tích hợp cơ cấu nâng hạ cần cẩu và hiển thị	02 động cơ Servo MG90S + Màn hình OLED HUD SSD1306	**Đạt 100%**

3.7. Kết luận chương 3

Chương 3 đã chứng minh sự thành công của đề tài bài tập lớn thông qua các sản phẩm kỹ thuật cụ thể: từ việc đấu nối phần cứng chính xác theo sơ đồ nguyên lý, xây dựng các tầng phần mềm nhúng tối ưu cấp thanh ghi không dùng HAL, đến việc vượt qua toàn bộ 7 kịch bản kiểm thử bơm lỗi khắt khe. Mô hình hoạt động ổn định, minh

chứng rõ nét cho năng lực làm chủ kiến trúc vi điều khiển và mạng truyền thông công nghiệp của sinh viên chuyên ngành Viễn thông.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Các kết quả chính đã hoàn thành của đề tài

Qua quá trình nghiên cứu và thực nghiệm bài tập lớn môn Kỹ thuật Vi xử lý, nhóm sinh viên đã đạt được các kết quả nổi bật sau:

1. **Hoàn thành thiết kế và chế tạo mô hình xe phân tán Automotive Gateway:** Tích hợp Gateway TCU (ESP32), Body/Safety ECU (STM32F103), mạng truyền thông CAN Bus tốc độ cao (500 kbps), hệ thống lái 4 bánh cầu H, cơ cấu cần cẩu gắp 2-DOF, còi cảnh báo và màn hình HUD đồ họa OLED.
2. **Hiện thực hóa kiến trúc FOTA Dual-Bank Rollback tự phục hồi:**
   - Quy hoạch bộ nhớ Flash 128KB tối ưu thành 3 phân vùng độc lập (Bootloader 16KB, App1 56KB, App2 56KB).
   - Ứng dụng vùng nhớ phi bốc hơi Option Bytes Data0 (0x1FFFF804) tạo cơ chế hoán đổi phân vùng nguyên tử an toàn tuyệt đối.
   - Xây dựng giải thuật truyền khối trượt Burst Chunk 256 bytes, kiểm tra tính toàn vẹn kép bằng CRC16-Modbus cấp gói và toàn cục.
3. **Đáp ứng các tiêu chí an toàn kỹ thuật của vi xử lý:**
   - Xây dựng cơ chế tiền điều kiện an toàn (*Pre-condition Check*), từ chối lệnh nạp firmware khi xe đang chuyển động.
   - Thử nghiệm đạt yêu cầu kịch bản mất nguồn đột ngột (*Power-cut Fault Injection*), khẳng định khả năng tự phục hồi 100% về phiên bản firmware ổn định mà không bị brick vi điều khiển.
4. **Triển khai lập trình nhúng trực tiếp cấp thanh ghi:** Toàn bộ mã nguồn trên STM32 được viết bằng C thuần với thư viện vi mô tiêu chuẩn SPL và can thiệp thanh ghi trực tiếp, tuyệt đối không dùng thư viện HAL, biên dịch độc lập bằng Makefile và GCC Toolchain.

2. Các mặt còn hạn chế của mô hình

- Gói firmware truyền qua mạng Wi-Fi và CAN Bus hiện mới chỉ được kiểm tra toàn vẹn bằng mã CRC, chưa được mã hóa đối xứng (AES-128/256) hay ký số xác thực nguồn gốc để ngăn ngừa nguy cơ bị can thiệp dữ liệu (*Man-in-the-Middle Attack*).
- Giao thức truyền thông trên mạng CAN hiện tại là giao thức do nhóm tự định nghĩa, chưa mở rộng theo chuẩn chẩn đoán quốc tế chuyên ngành ô tô UDS on CAN (ISO 14229).

- Hệ thống truyền nạp toàn bộ tệp nhị phân (*Full Image OTA*), chưa áp dụng giải thuật cập nhật vi sai (*Delta OTA*) để tối ưu hơn nữa dung lượng tải nạp.

3. Hướng mở rộng và phát triển tiếp theo

- Chuẩn hóa giao thức truyền nhận firmware theo tiêu chuẩn chẩn đoán ô tô quốc tế UDS on CAN (ISO 14229 / ISO 15765-2).
- Tích hợp giải thuật nén và so lệch nhị phân Delta OTA để chỉ truyền các byte dữ liệu khác biệt, rút ngắn thời gian nạp xuống dưới 5 giây.
- Bổ sung thuật toán mật mã phần cứng AES-GCM và kiểm tra chữ ký số công khai ECDSA để bảo đảm an toàn thông tin đầu cuối.

TÀI LIỆU THAM KHẢO

[1] Bộ môn Kỹ thuật Vi xử lý – Khoa Viễn thông 1, *Bài giảng Kỹ thuật Vi xử lý*, Học viện Công nghệ Bưu chính Viễn thông, Hà Nội, 2024. [2] Robert Bosch GmbH, *CAN Specification Version 2.0*, Postfach 50, D-7000 Stuttgart 1, 1991. [3] International Organization for Standardization, *Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling*, ISO Standard 11898-1:2015, 2015. [4] STMicroelectronics, *STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced Arm-based 32-bit MCUs Reference Manual (RM0008)*, Rev 21, 2021. [5] STMicroelectronics, *Flash programming manual for STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced Arm-based 32-bit MCUs (PM0075)*, Rev 9, 2021. [6] STMicroelectronics, *Application Note AN2606: STM32 microcontroller system memory boot mode*, Rev 54, 2023. [7] ARM Limited, *Cortex-M3 Technical Reference Manual (Revision r2p1)*, ARM DDI 0337I, 2010. [8] Joseph Yiu, *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, 3rd Edition, Newnes, Elsevier, 2013. [9] Espressif Systems, *ESP32 Technical Reference Manual (Version 5.1)*, 2024. [10] Texas Instruments, *SN65HVD230 3.3-V CAN Transceivers Datasheet (SLOS398I)*, Revised 2018. [11] Khoa Kỹ thuật Điện tử 1 – Học viện Công nghệ Bưu chính Viễn thông, *Báo cáo đề tài NCKH Sinh viên 2025: Nghiên cứu và ứng dụng phương pháp Delta OTA cập nhật phần mềm từ xa trong mạng LoRa*, Hà Nội, 2025. [12] Khoa Kỹ thuật Điện tử 1 – Học viện Công nghệ Bưu chính Viễn thông, *Báo cáo đề tài NCKH Sinh viên 2025: Nghiên cứu và phát triển robot Micromouse giải mê cung tự động*, Hà Nội, 2025.